# Guion de exposición de 5 minutos

Diapositivas 9–17

## Cálculo de tiempo

Se toma una velocidad media de habla de 140 palabras por minuto. Por tanto:

$$5 \text{ min} \times 140 \text{ palabras/min} = 700 \text{ palabras}$$

El reparto se ajusta a la complejidad de cada diapositiva. Las diapositivas de control y esquiva tienen algo más de tiempo; cierre y resultados son más breves.

| Diap. | Tema | Tiempo | Palabras objetivo |
|---|---|---|---|
| 9 | Exportación e inferencia | 35 s | 82 |
| 10 | Procesado de máscaras | 35 s | 82 |
| 11 | Seguimiento del circuito | 30 s | 70 |
| 12 | Esquiva de cajas | 40 s | 93 |
| 13 | Control en Jetson | 40 s | 93 |
| 14 | Arquitectura general | 35 s | 82 |
| 15 | Resultados de prueba real | 30 s | 70 |
| 16 | Conclusiones | 40 s | 93 |
| 17 | Cierre | 15 s | 35 |
| | **Total** | **300 s** | **700** |

## Guion por diapositiva

### Diapositiva 9 – Exportación e inferencia

Una vez entrenado el modelo, el siguiente paso es prepararlo para ejecución en el sistema real. Para eso se exporta a formato ONNX desde el notebook completo de entrenamiento. En el runtime, el modelo se carga desde la carpeta de modelos de `socket_vision` y se ejecuta con OpenCV DNN. Cada frame se redimensiona, normaliza y pasa por la red para obtener dos máscaras binarias.

| Dato que pueden preguntar | Respuesta corta |
|---|---|
| Entrada del modelo | Cada frame se redimensiona a `256 x 256 px` antes de la inferencia. |
| Video recibido | La configuración de stream usa `512 x 288` a `12 FPS`. |
| Modelo cargado | `socket_vision/models/line_segmentation/line_model.onnx`. |
| Normalización | Escala `1/255`; se genera un blob RGB para OpenCV DNN. |
| Salida de red | Dos canales: canal 0 = circuito, canal 1 = caja. |

### Diapositiva 10 – Procesado de máscaras

El modelo devuelve una imagen aproximada de dónde está el circuito y dónde están las cajas. Esa salida se convierte en dos máscaras sencillas: una para la zona por la que puede avanzar el coche y otra para los obstáculos. En código, esto se implementa aplicando una sigmoide a la salida de la red y después un umbral para decidir qué píxeles pertenecen a cada clase. Luego se limpia la máscara del circuito para quitar puntos sueltos y rellenar pequeños huecos. Con esa imagen más estable, el programa mira la parte inferior del frame y calcula el centro de la zona transitable.

| Parámetro | Valor usado |
|---|---|
| Umbral general | 0.50. |
| Umbral circuito | 0.62. |
| Umbral caja | 0.50. |
| Limpieza morfológica | Kernel 5; cierre 2 iteraciones; apertura 1. |
| Filtro de ruido circuito | Componentes menores de 220 px se descartan. |
| ROI del centro del circuito | Del 50 % al 90 % de la altura de la imagen. |

## Diapositiva 11 – Lógica de seguimiento del circuito

Para seguir el circuito, el sistema compara el centro de la zona segmentada con el centro de la imagen. Si el centro del circuito aparece a la izquierda o a la derecha, se genera un error lateral normalizado. Ese error se transforma en un valor de *steering*. Cuando el error es pequeño, el coche sigue recto. Si no se detecta zona transitable suficiente, la política de seguridad permite detener el coche.

| Dato de control | Valor / fórmula |
|---|---|
| Error lateral | center_offset_px = centro_circuito - centro_imagen. |
| Error normalizado | offset_px / (ancho / 2). |
| Suavizado | Media exponencial con alpha = 0.35. |
| Zona muerta | 0.02; por debajo se considera centrado. |
| Ganancia de giro | Kp = 3.2 y ganancia curva 2.6. |
| Pérdida de circuito | Política stop: steering 0, throttle 0. |

## Diapositiva 12 – Esquiva de cajas

La evitación de obstáculos se implementa en autonomy.py. El sistema analiza la máscara de cajas en la zona frontal, porque es la parte que afecta directamente a la trayectoria. De cada obstáculo se estima posición, área y urgencia. Si la caja aparece a la izquierda, se gira hacia la derecha; si aparece a la derecha, se gira hacia la izquierda. Además, cuanto mayor es el riesgo, más se reduce la velocidad, hasta llegar a parada si es necesario.

| Dato de esquiva | Valor usado |
|---|---|
| ROI frontal de obstáculos | Desde el 35 % de la altura hacia abajo. |
| Zona horizontal válida | Entre 20 % y 80 % del ancho. |
| Mínimo de píxeles caja | 180 px; componente mínimo 260 px. |
| Tamaño mínimo componente | Anchura 36 px, altura 26 px. |
| Area mínima / emergencia | 0.008 / 0.02 del área del frame. |
| Giro de esquiva | Entre 0.28 y 0.68; retención 0.45 s. |

## Diapositiva 13 – Control del coche en Jetson

El servidor de control en la Jetson recibe comandos auto_drive por socket. Cada comando incluye dirección y velocidad, es decir, *steering* y *throttle*. El archivo control_server.py aplica esos valores usando NvidiaRacecar. También incluye calibración de dirección y límites para no mandar valores peligrosos. El control manual tiene prioridad temporal sobre el automático, y el watchdog detiene el coche si dejan de llegar comandos.

| Dato del servidor | Valor usado |
|---|---|
| Puerto de control | 5060. |
| Comando automático | JSON por línea con cmd=auto_drive, steering y throttle. |
| Watchdog automático | Si no llegan comandos en 1 s, detiene el coche. |
| Prioridad manual | 1.8 s; parada manual mantiene prioridad 3 s. |
| Throttle automático | Base 0.55; máximo 0.60. |
| Límites | *steering* y *throttle* se saturan en [-1, 1]. |

## Diapositiva 14 – Arquitectura general

La arquitectura se separa en dos partes principales. `circuito_segmentation` contiene todo lo relacionado con dataset, entrenamiento, pruebas y exportación del modelo. `socket_vision` contiene la ejecución en tiempo real: visión, comunicación, control y procesos de Jetson. El PC se usa para entrenar y preparar el modelo. La Jetson captura vídeo y ejecuta el control sobre el coche. El script `main.sh` sincroniza archivos y lanza el flujo completo.

| **Parte** | **Dato rápido** |
|---|---|
| Jetson | Captura cámara y ejecuta `sender.py` + `control_server.py`. |
| Portátil | Recibe video, ejecuta segmentación y calcula comandos. |
| Video | Jetson → portátil por UDP, puerto `5050`. |
| Control | Portátil → Jetson por socket TCP, puerto `5060`. |
| IPs de configuración | Jetson `192.168.1.34`; receptor `192.168.1.43`. |
| Lanzamiento | `socket_vision/conexion/scripts/main.sh`. |

## Diapositiva 15 – Resultados de prueba real

En la prueba real se validó que el sistema estaba integrado de extremo a extremo: captura de vídeo, segmentación, cálculo del centro del circuito, detección de cajas y envío de comandos al coche. La parte positiva es que no se quedó solo en un modelo entrenado, sino que funcionó dentro del flujo real con Jetson y control. Lo que vimos como principales limitantes fue la parte física de visión, por el campo de visión reducido o la colocación adelantada de la cámara, y la latencia generada al mandar el vídeo al portátil, procesarlo y devolver el comando a la Jetson.

| **Dato de prueba/modelo** | **Valor** |
|---|---|
| Prueba real | Integración validada: cámara, segmentación, comunicación y control. |
| Limitante de visión | Campo de visión reducido o cámara demasiado adelantada. |
| Limitante de latencia | Vídeo Jetson → portátil y comando portátil → Jetson. |
| Dataset propio | `57` imágenes y `100` anotaciones. |
| Resolución anotada | `640 x 360 px`. |
| Partición | `45` entrenamiento, `12` validación. |
| Clases | `circuito` y `caja`. |
| Arquitectura | U-Net ligera con `7.763.074` parámetros entrenables. |
| Entrada / salida | Entrada `3 x 256 x 256`; salida `2` canales. |
| Entrenamiento | `BCEWithLogits + Dice Loss`, optimizador `Adam`. |
| Batch size | `8`. |
| Mejor checkpoint | Época `78`; Dice de validación `0.9519`. |
| Uso final | Exportado a `ONNX` para inferencia con OpenCV DNN. |

## Diapositiva 16 – Conclusiones

Como conclusión, la segmentación semántica encaja bien con este problema porque devuelve información por píxel y permite tomar decisiones simples. Con solo dos clases, circuito y caja, se cubre la tarea principal. La U-Net ligera reduce el coste de cómputo y facilita la inferencia. La prueba real también dejó claro que los mayores límites actuales no son solo del modelo: influyen mucho el campo de visión de la cámara, su colocación y la latencia del flujo distribuido. Como mejoras futuras, priorizaría recolocar o cambiar la cámara y reducir latencia; después ampliar datos, ajustar umbrales y estudiar DeepLabv3+ o TensorRT.

| Posible pregunta | Respuesta breve |
|---|---|
| Por qué segmentación | Porque da información por píxel, no solo una caja o etiqueta global. |
| Por qué dos clases | Bastan para decidir por dónde avanzar y qué obstáculo evitar. |
| Por qué U-Net ligera | Menor coste que DeepLabv3+ con ResNet50 para un robot con recursos limitados. |
| Qué no se midió | No hay métrica formal de latencia/FPS de inferencia en log experimental. |
| Qué limitó la prueba | Campo de visión/posición de cámara y latencia Jetson-portátil-Jetson. |
| Futuras mejoras | Mejor cámara o posición, menos latencia, más datos, TensorRT y evaluación formal. |

## Diapositiva 17 – Cierre

La idea final es que se ha construido una tubería completa: captura, etiquetado, entrenamiento, exportación, inferencia y control. No es solo un modelo entrenado, sino una integración funcional con Jetson y con el control real del coche.